formulas

1. Core Computational Pillars

- **Sample Statistics**
  - **Mean Vector:** $\bar{\mathbf{x}} = \frac{1}{N}\sum_{n=1}^{N}\mathbf{x}_n$
  - *In Plain English:* The average position or "center of mass" of your data points in space.
  - **Covariance Matrix:** $\mathbf{S} = \frac{1}{N}\sum_{n=1}^{N}(\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$
  - *In Plain English:* Measures how much dimensions vary together; the diagonal is the variance of each dimension.
- **Maximum Likelihood (Gaussian)**
  - **Log-Likelihood:** $\ln p(\mathbf{x}|\mu, \sigma^2) = -\frac{N}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum(x_n - \mu)^2$
  - **MLE Mean:** $\hat{\mu} = \frac{1}{N}\sum x_n$
  - **MLE Variance:** $\hat{\sigma}^2 = \frac{1}{N}\sum(x_n - \hat{\mu})^2$
  - *In Plain English:* Parameters that make the observed data the most probable outcome under a normal distribution.
- **Bayesian Bernoulli-Beta Update**
  - **Posterior Form:** $\mathrm{Beta}(a + m, b + l)$ where m = successes, l = failures.
  - **Posterior Mean:** $\frac{a+m}{a+m+b+l}$
  - **Posterior Mode:** $\frac{a+m-1}{a+m+b+l-2}$ (valid when $a + m > 1$ and $b + l > 1$)
  - *In Plain English:* Your initial belief (prior) updated by counting real-world "wins" and "losses".
- **Linear Regression**
  - **Normal Equations:** $\mathbf{w} = (\Phi^T\Phi)^{-1}\Phi^T\mathbf{t}$
  - **Ridge (L2) Solution:** $\mathbf{w} = (\Phi^T\Phi + \lambda\mathbf{I})^{-1}\Phi^T\mathbf{t}$

- *In Plain English:* Finding the best-fit line by minimizing squared errors, with Ridge adding a "penalty" to keep weights small and prevent over-fitting.

- **Bias-Variance Decomposition**
  - **Total Loss:** $(\text{Bias})^2 + \text{Variance} + \text{Noise}$
  - **Squared Bias:** $(\bar{y}(x) - h(x))^2$ where $h(x)$ is the truth.
  - **Variance:** $\frac{1}{K}\sum_{k=1}^{K}(y^{(k)}(x) - \bar{y}(x))^2$
  - *In Plain English:* Bias = error from wrong assumptions; Variance = sensitivity to dataset; Noise = irreducible.
- **Bayesian Linear Regression**
  - **Posterior Covariance:** $\mathbf{S}_N^{-1} = \alpha\mathbf{I} + \beta\mathbf{\Phi}^T\mathbf{\Phi}$
  - **Posterior Mean:** $\mathbf{m}_N = \beta\mathbf{S}_N\mathbf{\Phi}^T\mathbf{t}$
  - **Predictive Mean:** $\mu(x_*) = \mathbf{m}_N^T\phi(x_*)$
  - **Predictive Variance:** $\sigma_N^2(x_*) = \frac{1}{\beta} + \phi(x_*)^T\mathbf{S}_N\phi(x_*)$
  - *In Plain English:* Uncertainty in prediction comes from both intrinsic data noise $(1/\beta)$ and uncertainty in our weight estimates $(\mathbf{S}_N)$.
- **Model Evidence & Evidence Approximation**
  - **Log Marginal Likelihood (Approximation):** $\ln p(\mathcal{D}|\mathcal{M}) \approx \ln p(\mathcal{D}|\mathbf{w}_{\text{MAP}}) + M\ln\frac{\Delta w_{\text{posterior}}}{\Delta w_{\text{prior}}}$
  - **Effective Parameters:** $\gamma = \sum_i \frac{\lambda_i}{\alpha+\lambda_i}$ where λ_i are eigenvalues of $\beta\mathbf{\Phi}^T\mathbf{\Phi}$
  - α **Update:** $\alpha_{\text{new}} = \frac{\gamma}{\mathbf{m}_N^T\mathbf{m}_N}$
  - *In Plain English:* The "evidence" balances model complexity against fit quality; γ counts how many parameters are actually useful.

---

2. Classification Formulas

- **Binary Logistic Regression:** $p(C_1|\phi) = \sigma(\mathbf{w}^T\phi)$ where $\sigma(a) = \frac{1}{1+e^{-a}}$
  - *In Plain English:* Squeezes a linear score into a 0-to-1 probability range for classification.

- **Softmax (Multiclass):** $y_k(\mathbf{x}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$ where $a_k = \mathbf{w}_k^T \tilde{\mathbf{x}}$
  - *In Plain English:* A multiclass extension of the sigmoid that ensures all class probabilities sum to 1.
- **Binary Classification via Least Squares:** Fit $y(\mathbf{x}) = \mathbf{w}^T \tilde{\mathbf{x}}$ using normal equations, classify by thresholding at $y(\mathbf{x}) = 0.5$.
  - *Weakness:* Sensitive to outliers; does not produce calibrated probabilities unlike logistic regression.
- **Cross-Entropy Error (single point):** $E_n = -[t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$
  - **Gradient:** $\frac{\partial E_n}{\partial w_j} = (y_n - t_n)\phi_j(\mathbf{x}_n)$
  - *Key insight:* The gradient has the same form as least squares but with $y_n = \sigma(\mathbf{w}^T \phi)$.

---

3. Gradient Descent Formulas

- **Batch Gradient Descent:** $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$
- **Sequential (Online) Gradient Descent:** $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$
  - Process one data point at a time, update after each.
- **For Linear Regression** $y(x) = w_0 + w_1 x$:
  - $\frac{\partial E_n}{\partial w_0} = (y(x_n) - t_n)$
  - $\frac{\partial E_n}{\partial w_1} = (y(x_n) - t_n) \cdot x_n$
  - Update: $w_j \leftarrow w_j - \eta(y(x_n) - t_n)\phi_j(x_n)$
- **For Logistic Regression** $y = \sigma(\mathbf{w}^T \phi)$:
  - $\frac{\partial E_n}{\partial w_j} = (y_n - t_n)\phi_j$
  - Update: $w_j \leftarrow w_j - \eta(y_n - t_n)\phi_j$

---

4. Gaussian / Bayesian Update Formulas

- **Multivariate Gaussian PDF:**

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{D/2}|\mathbf{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\mathbf{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

- **Univariate Gaussian PDF:** $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$
- **Bayesian Update for Gaussian Mean** (known variance σ^2, prior $\mu \sim \mathcal{N}(\mu_0, \sigma_0^2)$):
  - **Posterior Precision:** $\frac{1}{\sigma_N^2} = \frac{1}{\sigma_0^2} + \frac{N}{\sigma^2}$
  - **Posterior Mean:** $\mu_N = \sigma_N^2\left(\frac{\mu_0}{\sigma_0^2} + \frac{\sum x_n}{\sigma^2}\right)$
  - For a single observation: $\mu_N = \sigma_N^2\left(\frac{\mu_0}{\sigma_0^2} + \frac{x}{\sigma^2}\right)$, $\frac{1}{\sigma_N^2} = \frac{1}{\sigma_0^2} + \frac{1}{\sigma^2}$
- **Conjugate Priors Quick Reference:**
  - Bernoulli likelihood → Beta prior
  - Gaussian likelihood (known variance) → Gaussian prior for mean
  - Gaussian likelihood (known mean) → Inverse-Gamma prior for variance

---

5. Information Theory

- **Entropy:** $H(x) = -\sum p(x)\ln p(x)$
- **KL Divergence:** $KL(p||q) = -\int p(x)\ln\frac{q(x)}{p(x)}dx \geq 0$ (always non-negative, zero iff $p = q$)
- **Kernel Trick:** $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T\phi(\mathbf{x}')$

---

6. Key Matrix Operations (Computational Shortcuts)

🔧 **2×2 Matrix Inversion:**
$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$
Determinant: $\det = ad - bc$

🔧 **3×3 Matrix Inversion:** Use cofactor expansion or Cramer's rule. Often given in the problem.
Determinant (3×3): $\det(A) = a(ei - fh) - b(di - fg) + c(dh - eg)$ for $A = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$

🔧 **Eigenvalues of 2×2 Matrix** $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$:
$\lambda = \frac{(a+d) \pm \sqrt{(a+d)^2 - 4(ad-bc)}}{2}$
PSD Check: All eigenvalues $\lambda \geq 0$.

🔧 **Outer Product (2D):** $\mathbf{v}\mathbf{v}^T$ where $\mathbf{v} = (v_1, v_2)^T$ gives $\begin{pmatrix} v_1^2 & v_1 v_2 \\ v_1 v_2 & v_2^2 \end{pmatrix}$
Useful for computing covariance matrices term by term.

---

7. Boundary Conditions & Common Pitfalls

- **MLE Variance Bias:** $\hat{\sigma}^2_{\text{MLE}}$ underestimates by factor $\frac{N-1}{N}$. Unbiased: $s^2 = \frac{1}{N-1} \sum (x_n - \bar{x})^2$.
- **Matrix Inversion Failure:** $\Phi^T \Phi$ is singular if $N < M$ or features are collinear. Ridge $(\lambda \mathbf{I})$ fixes this.
- **Beta Distribution:** $a, b > 0$. $\text{Beta}(1, 1)$ = uniform prior. Mode exists only when $a > 1$ and $b > 1$.
- **Sigmoid Properties:** $\sigma(0) = 0.5$, $\sigma(-a) = 1 - \sigma(a)$, $\frac{d\sigma}{da} = \sigma(a)(1 - \sigma(a))$.
- **Softmax Overflow Trick:** Subtract $\max(a_k)$ from all activations before computing $\exp$.
- **Ridge vs. No Regularization:** As $\lambda \to 0$, ridge → OLS. As $\lambda \to \infty$, all weights → 0.

8. Workflow Shortcuts: Step-by-Step Micro-Algos

A. Polynomial Design Matrix (Φ) Construction

1. Identify Degree M (e.g., $M = 2$ means features are $1, x, x^2$).
2. For each input x_n, create the row: $\phi(x_n)^T = [x_n^0, x_n^1, \ldots, x_n^M]$.
3. Stack the N rows to form the $N \times (M+1)$ matrix.

B. The 4-Step Ridge Regression Workflow

1. Compute $\mathbf{A} = \Phi^T \Phi$.
2. Add λ to every diagonal element: $\mathbf{A}_{\text{reg}} = \mathbf{A} + \lambda \mathbf{I}$.
3. Compute target vector $\mathbf{b} = \Phi^T \mathbf{t}$.
4. Solve $\mathbf{A}_{\text{reg}} \mathbf{w} = \mathbf{b}$ → $\mathbf{w} = \mathbf{A}_{\text{reg}}^{-1} \mathbf{b}$.

C. Bayesian Linear Regression Workflow

1. Build Φ (design matrix).
2. Compute $\mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^T \Phi$.
3. Invert to get $\mathbf{S}_N$.
4. Compute $\mathbf{m}_N = \beta \mathbf{S}_N \Phi^T \mathbf{t}$.
5. For prediction at x_*: mean $= \mathbf{m}_N^T \phi(x_*)$, variance $= 1/\beta + \phi(x_*)^T \mathbf{S}_N \phi(x_*)$.

D. Bayesian Update (Bernoulli-Beta)

1. Prior $\text{Beta}(a, b)$.
2. Count Successes (m) and Failures (l).
3. Posterior = $\text{Beta}(a + m, b + l)$.
4. Mean = $\frac{a+m}{a+m+b+l}$, Mode = $\frac{a+m-1}{a+m+b+l-2}$.

E. Bias-Variance Check

1. Average the predictions: $\bar{y} = \frac{1}{K} \sum y^{(k)}$.
2. $(\text{Bias})^2 = (\bar{y} - \text{truth})^2$.
3. $\text{Var} = \frac{1}{K} \sum (y^{(k)} - \bar{y})^2$.
4. Total Expected Loss = $(\text{Bias})^2 + \text{Var} + \sigma^2_{\text{noise}}$.

F. Model Evidence Approximation Workflow

1. Compute MAP log-likelihood: $\ln p(\mathcal{D}|\mathbf{w}_{\text{MAP}})$.
2. Complexity penalty: $M \cdot \ln \frac{\Delta w_{\text{posterior}}}{\Delta w_{\text{prior}}}$ (negative, penalizes complexity).
3. Sum: $\ln p(\mathcal{D}|\mathcal{M}) \approx \text{step } 1 + \text{step } 2$.
4. Compare models: higher log evidence = preferred model.

G. Evidence Approximation (α Update)

1. Compute eigenvalues λ_i of $\beta \Phi^T \Phi$.
2. $\gamma = \sum_i \frac{\lambda_i}{\alpha + \lambda_i}$.
3. $\alpha_{\text{new}} = \frac{\gamma}{\mathbf{m}_N^T \mathbf{m}_N}$.

H. Sequential Gradient Descent (Linear Regression)

1. Initialize $\mathbf{w} = \mathbf{0}$.
2. For each data point (x_n, t_n):
   - Compute $y_n = \mathbf{w}^T \phi(x_n)$.
   - Compute error $e_n = y_n - t_n$.
   - Update $w_j \leftarrow w_j - \eta \cdot e_n \cdot \phi_j(x_n)$.

I. Softmax Computation

1. Compute activations $a_k(\mathbf{x}) = \mathbf{w}_k^T \tilde{\mathbf{x}}$ for each class.
2. Exponentiate: $\exp(a_k)$ for each class.
3. Normalize: $p(C_k|\mathbf{x}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$.
4. Predicted class = $\arg\max_k p(C_k|\mathbf{x})$.